

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Mock Interview (Low)
Assessment Tool Weightage: 30%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Interviewer: 'Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?'	BL3 – Apply	5
2	Interviewer: 'What is the difference between heuristic optimization and cost-based optimization? When would you use each?'	BL4 – Analyze	5
3	Interviewer: 'How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.'	BL3 – Apply	5
4	Interviewer: 'What are ACID properties? Give a real-world example where each property is critical.'	BL3 – Apply	5
5	Interviewer: 'Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?'	BL4 – Analyze	5
6	Interviewer: 'What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.'	BL4 – Analyze	5
7	Interviewer: 'Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?'	BL4 – Analyze	5
8	Interviewer: 'Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?'	BL3 – Apply	5
9	Interviewer: 'What is a serializability schedule? How do you test for conflict serializability using a precedence graph?'	BL4 – Analyze	5

10	Interviewer: 'Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?'	BL4 – Analyze	5
----	--	---------------	---

Code of Conduct:

I Kadhirezhilan. D(192511228) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

1. Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?

Answer: Query processing converts an SQL query into an efficient executable plan. The main stages are **parsing and validation** → **translation** → **query rewriting** → **optimization** → **execution**. The parser checks syntax and semantics and converts the SQL into an internal representation. The optimizer generates alternative plans, such as sequential scan, index scan, different join orders, and join algorithms. It estimates their costs using statistics such as number of tuples, pages, selectivity, available indexes, CPU, memory, and disk I/O. The plan with the lowest estimated cost is selected and executed. For example, for a query filtering a large table on an indexed column, an index scan may be cheaper than a full table scan. Thus, query optimization aims to obtain the required result with minimum practical execution cost.

2. Difference between heuristic and cost-based optimization. When is each used?

Answer: **Heuristic optimization** applies predefined rules to improve a query, such as performing selections and projections early and reducing intermediate results. It is fast and simple but does not guarantee the cheapest plan. **Cost-based optimization** generates alternative execution plans and estimates their costs using database statistics, then chooses the plan with the lowest estimated cost. For example, a DBMS may compare nested-loop, hash, and merge joins and select the cheapest one. Heuristics are useful for quick transformations and reducing the optimization search space, while cost-based optimization is preferred for complex production queries. Modern DBMSs commonly combine both approaches.

3. How would you implement database connectivity in a real project? Explain JDBC and ODBC.

Answer: Database connectivity involves **loading/configuring the driver, establishing a connection, creating a statement or prepared statement, executing SQL, processing results, handling exceptions, and closing resources**. In Java, **JDBC (Java Database Connectivity)** is the standard API for communicating with relational databases through JDBC drivers. **ODBC (Open Database Connectivity)** is a general database connectivity standard usable across programming environments through ODBC drivers. JDBC is therefore the

natural choice for a Java application, whereas ODBC is useful when a system requires a language-independent database connectivity interface. In a real project, prepared statements, connection pooling, proper exception handling, and secure credential management should also be used.

4. What are ACID properties? Give real-world examples.

Answer: ACID ensures reliable transaction processing:

- **Atomicity:** A transaction is all-or-nothing. In a bank transfer, money must either be deducted from Account A **and** added to Account B, or neither operation should occur.
- **Consistency:** A transaction must preserve database rules and constraints, keeping the database in a valid state.
- **Isolation:** Concurrent transactions should not incorrectly interfere. For example, two simultaneous ATM withdrawals must not both use the same outdated balance.
- **Durability:** Once a transaction commits, its changes must survive failures such as power loss or system crashes.

Therefore, ACID is essential in banking, healthcare, reservation, and other systems where incorrect or lost transactions can cause serious consequences.

5. Explain dirty read, lost update, and unrepeatable read. How does 2PL solve them?

Answer: A **dirty read** occurs when a transaction reads data written by another transaction before it commits. A **lost update** occurs when concurrent transactions overwrite each other's updates. An **unrepeatable read** occurs when the same transaction reads a record twice but gets different values because another transaction modified and committed it between the reads. **Two-Phase Locking (2PL)** controls these conflicts through two phases: the **growing phase**, where locks are acquired, and the **shrinking phase**, where locks are released. Basic 2PL guarantees conflict serializability. **Strict 2PL**, which holds exclusive locks until commit/abort, additionally prevents other transactions from using uncommitted writes and simplifies recovery. Thus, 2PL provides controlled concurrent execution and reliable isolation.

6. What is deadlock? Explain detection and two prevention strategies.

Answer: A **deadlock** occurs when transactions wait indefinitely for resources held by each other. For example, T1 holds A and waits for B, while T2 holds B and waits for A. For **deadlock detection**, the DBMS creates a **Wait-For Graph**, where nodes represent transactions and edges represent waiting relationships. A cycle indicates a deadlock; the DBMS can select a victim transaction, roll it back, release its locks, and allow the others to continue. Two prevention techniques are **resource ordering**, where transactions must acquire resources in a fixed order, and **timestamp-based schemes such as Wait-Die or Wound-Wait**, which determine whether a transaction waits or rolls back. Prevention avoids deadlocks, while detection identifies and resolves them after they occur.

7. Compare Immediate Update and Deferred Update recovery.

Answer: In **Deferred Update**, changes are not applied to the database until the transaction commits. Therefore, if the transaction fails before commit, its database changes need not be undone. It provides relatively simple recovery and is suitable when delayed application of updates is acceptable. In **Immediate Update**, database changes may reach storage before commit, so recovery must use logging to **UNDO uncommitted changes** and, where required, **REDO committed changes**. Immediate update provides greater flexibility but requires more sophisticated recovery. Therefore, deferred update favors simpler recovery, while immediate update is preferred when the system needs flexibility to write changes before commit and has robust logging and recovery mechanisms.

8. Explain Shadow Paging and compare it with log-based recovery.

Answer: **Shadow Paging** maintains a stable **shadow page table** and a current page table. When a page is modified, the DBMS writes the new version to a different page instead of overwriting the original. The shadow pages remain unchanged until commit. At commit, the system switches the appropriate page-table reference to the new version. If a failure occurs before commit, the old shadow structure remains available. Its advantages are simple and fast recovery and reduced need for traditional undo processing. However, it can cause **storage overhead, fragmentation, page management, and garbage collection**, and it is less attractive

for high-concurrency workloads. **Log-based recovery** uses write-ahead logging with UNDO/REDO and is generally more flexible and scalable for modern DBMSs.

9. What is serializability? How is conflict serializability tested using a precedence graph?

Answer: A **serial schedule** executes transactions one after another, while a **serializable schedule** may interleave operations but produces an equivalent result to some serial execution. To test **conflict serializability**, construct a **precedence graph**: create one node for each transaction and add an edge $T_i \rightarrow T_j$ when conflicting operations of T_i occur before those of T_j . Conflicts include **Read–Write, Write–Read, and Write–Write** operations on the same data item; Read–Read is not a conflict. After constructing the graph, check for cycles. **If the graph is acyclic, the schedule is conflict serializable; if it contains a cycle, it is not conflict serializable.** This provides a systematic method for validating concurrent schedules.

10. Explain Basic, Strict, and Rigorous 2PL. Which is most commonly used and why?

Answer: **Basic 2PL** divides execution into a growing phase, where locks can be acquired, and a shrinking phase, where locks can be released. It guarantees conflict serializability but may allow cascading rollback. **Strict 2PL** holds **exclusive locks until commit or abort**, preventing other transactions from accessing uncommitted writes and simplifying recovery. **Rigorous 2PL** holds **both shared and exclusive locks until commit or abort**, providing an even stronger locking discipline but potentially reducing concurrency. In practical DBMS environments, **Strict 2PL is commonly preferred** because it provides a strong balance between serializability, isolation, and recovery simplicity without requiring every shared lock to remain until transaction completion.